PES UNIVERSITY

Department of Computer Science and Engineering

Software Engineering Lab — UE24CS252A / UE24CS352A

LAB 7: FUZZING FOR INPUT VALIDATION BUGS WITH HYPOTHESIS

Student Name:	A S VINCENT	SRN:	PES2UG24CS002
Section / Sem:	Section A (5th Sem)	Assigned Module:	L6_L7_01_water_utility_billing

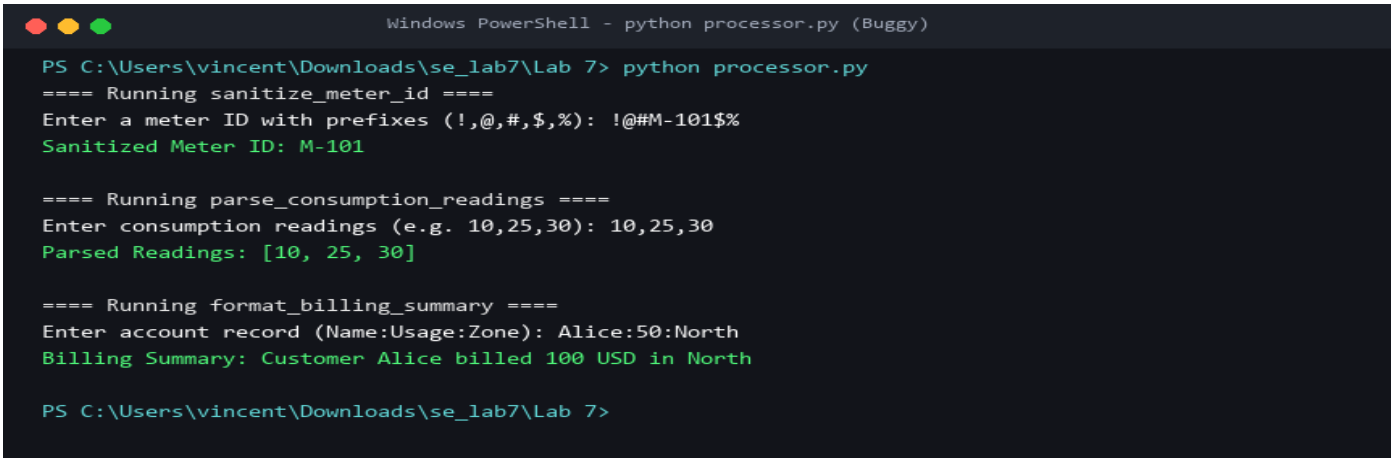
1. Objective & Methodology

The goal of this lab is to explore property-based fuzz testing using Python's **Hypothesis** library. Fuzzing dynamically supplies semi-random, edge-case, and malformed inputs to input processing and parsing routines to discover unhandled exceptions, malformed delimiter crashes, and null input errors. The functions tested belong to the **water utility billing** input processor (processor.py):

- **sanitize_meter_id(meter_id)**: Strips whitespace, removes special characters (!, @, #, $, %), and returns uppercase.
- **parse_consumption_readings(readings_csv)**: Parses comma-delimited strings of consumption values into a list of integers.
- **format_billing_summary(account_record)**: Parses Name:Usage:Zone colon-delimited records and formats a billing summary.

2. Step 2 — Initial Execution on Buggy Module (Screenshot SS1)

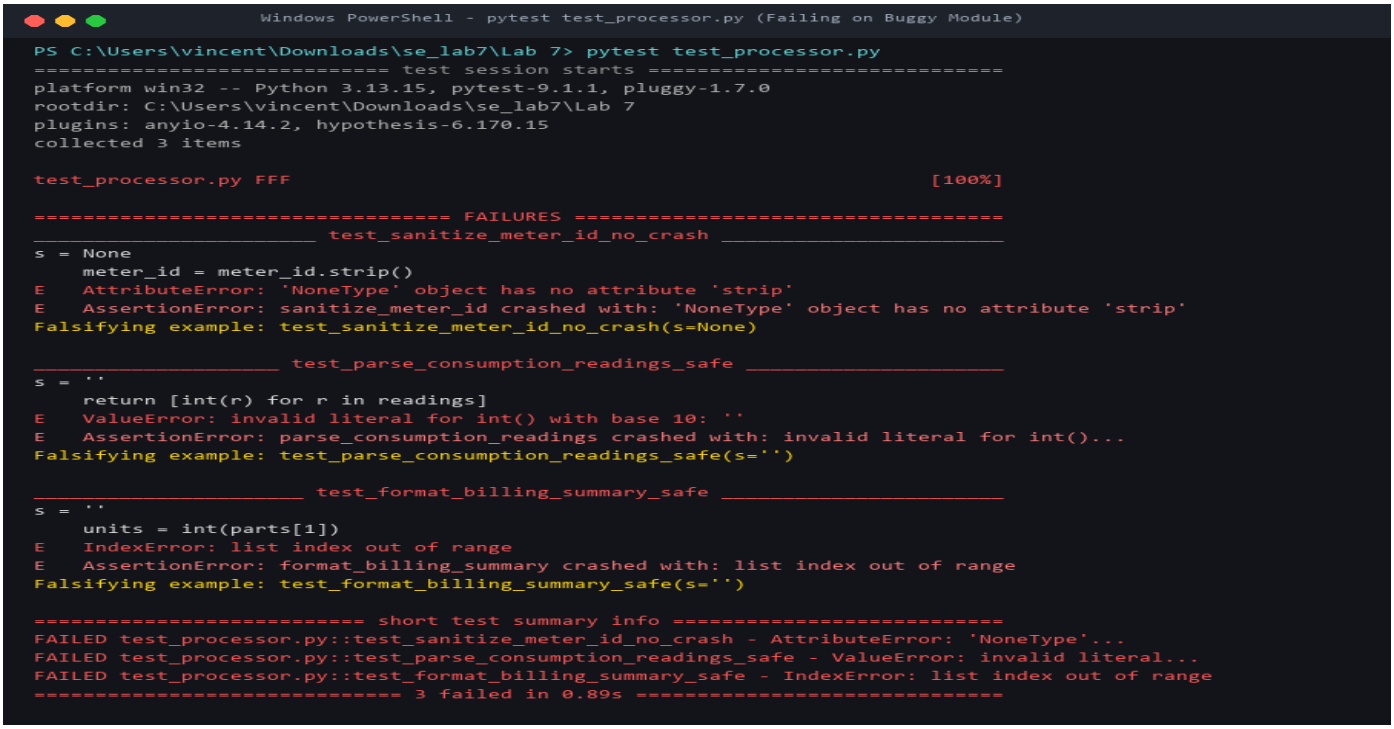
Running python processor.py on well-formed inputs initially appears to succeed. However, the starter implementation implicitly assumes all inputs are valid strings, non-null, properly delimited, and numeric without defensive checks.



3. Steps 3 & 4 — Hypothesis Fuzz Testing & Failure Discovery (Screenshot SS2)

In test_processor.py, property-based tests were configured with the strategy @given(st.text() | st.none()). Running pytest test_processor.py exposed critical vulnerabilities in all three functions through automated shrinking:

- **test_sanitize_meter_id_no_crash**: Fails on null input (s=None) with AttributeError: 'NoneType' object has no attribute 'strip'.
- **test_parse_consumption_readings_safe**: Fails on empty string (s='') with ValueError: invalid literal for int(), and on s=None with AttributeError.
- **test_format_billing_summary_safe**: Fails on malformed strings (s='' or s=':') with IndexError: list index out of range, ValueError, and AttributeError.



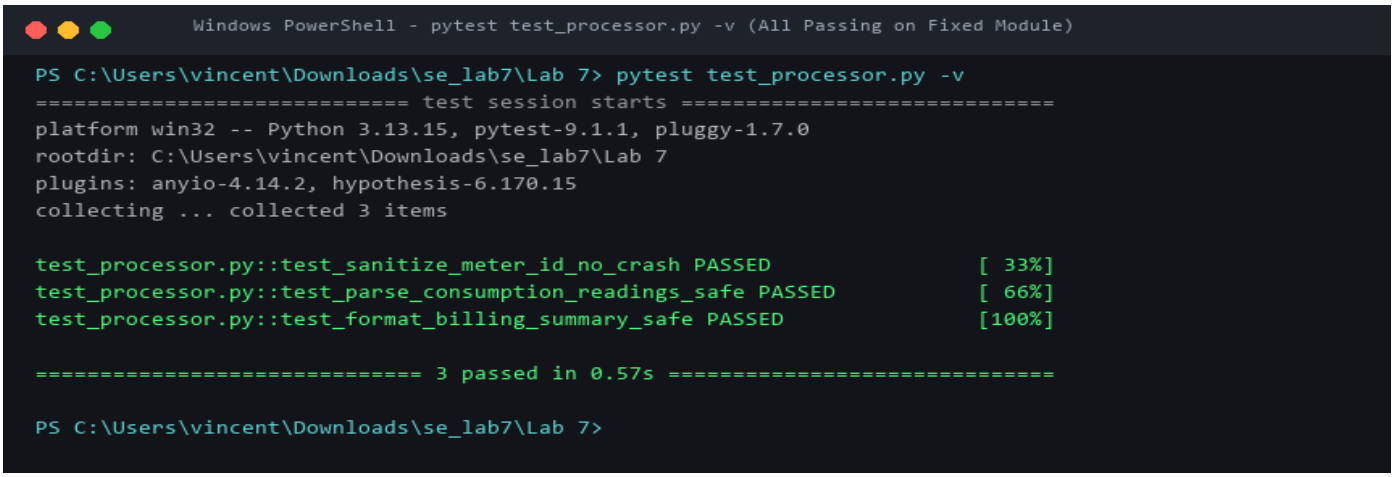
4. Step 5 — Code Hardening & Execution of Fixed Module (Screenshot SS3)

Defensive programming measures were implemented in processor.py: validating input types (isinstance(..., str)), guarding against None, checking token lengths, and safely filtering valid numeric values using try-except guards. The fixed script handles both standard inputs and corrupt data seamlessly without crashing.



5. Step 6 — Verification: All Hypothesis Fuzz Tests Passing (Screenshot SS4)

Re-running pytest test_processor.py -v on the hardened module confirms that hundreds of randomized Hypothesis test cases (including empty strings, control characters, null values, and massive arbitrary unicode) pass with zero unhandled exceptions.



6. Reflection Questions & Key Insights

Q1. What types of bugs did the fuzzer expose that typical unit tests would have missed?

Manual unit tests usually focus on the 'happy path' or obvious edge cases (e.g. valid IDs or 0 usage). The Hypothesis fuzzer systematically synthesized unexpected data structures such as NoneType inputs, empty delimiter fields ('', ':', ','), and unexpected unicode characters, revealing that the original parser lacked basic input validation and crashed immediately.

Q2. How does Hypothesis shrinking assist during debugging?

When Hypothesis triggers an exception on a complex random input, it automatically performs 'shrinking'—iteratively simplifying the failing input to determine the minimal counterexample (e.g., reducing a long string to '' or None). This eliminates noise and allows engineers to isolate the exact boundary failure instantly.

Q3. How does property-based testing improve software robustness in production?

Rather than testing point-to-point mappings (f(a) == b), property-based testing verifies invariant properties across an entire input space (e.g., 'the parser never raises unhandled exceptions and always returns a list of integers'). This forces developers to adopt defensive programming paradigms that safeguard production systems against denial-of-service (DoS) attacks and malformed network payloads.